

APOSTILA OFICIAL DE PROJETO

# C Larp

## Do primeiro código à engenharia da linguagem

Introdução, códigos simples, design de software,  
bibliotecas, compilador e Bilarp Linux

Versão 0.3 - documento de projeto

Público: iniciantes, estudantes e colaboradores

BILARP LINUX | 22 DE JULHO DE 2026

# Sobre este documento

Esta apostila ensina a proposta da C Larp desde o primeiro código e também registra as decisões de engenharia necessárias para construir um compilador e ferramentas reais para o Bilarp Linux.

Estado do projeto: a linguagem ainda será implementada. Exemplos são contratos de design, não evidência de software pronto.

| Item           | Definição                                                    |
|----------------|--------------------------------------------------------------|
| Público        | Iniciantes, estudantes e colaboradores técnicos              |
| Escopo         | Programas Linux x86-64 e ferramentas do Bilarp               |
| Fora do escopo | Kernel próprio, banco de dados próprio e hardware irrestrito |
| Backend        | LLVM e LLD; frontend inicial em Rust                         |
| Licença        | A definir pelo projeto antes da distribuição pública         |

## Princípio editorial

Cada conceito aparece primeiro em linguagem simples. Em seguida surgem exemplos, limites e detalhes técnicos. Recursos planejados são identificados por estágio para que ninguém confunda intenção com implementação.

# Sumário

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>Sobre este documento</b>                        | <b>2</b>  |
| Princípio editorial . . . . .                      | 2         |
| <b>Como usar esta apostila</b>                     | <b>9</b>  |
| Caminho de leitura . . . . .                       | 9         |
| Símbolos didáticos . . . . .                       | 9         |
| Maturidade dos recursos . . . . .                  | 9         |
| <b>PARTE I - O projeto em linguagem simples</b>    | <b>10</b> |
| <b>1. O que estamos construindo</b>                | <b>10</b> |
| 1.1 O que é uma linguagem de programação . . . . . | 10        |
| 1.2 O que é um compilador . . . . .                | 11        |
| 1.3 O que é uma distribuição Linux . . . . .       | 11        |
| 1.4 O que não vamos prometer . . . . .             | 11        |
| <b>2. Conhecendo um código C Larp</b>              | <b>12</b> |
| 2.1 Explicação linha por linha . . . . .           | 12        |
| 2.2 Espaços e indentação . . . . .                 | 12        |
| 2.3 Comentários . . . . .                          | 12        |
| <b>3. Mostrando mensagens</b>                      | <b>13</b> |
| 3.1 O que significa use . . . . .                  | 13        |
| 3.2 Texto entre aspas . . . . .                    | 13        |
| 3.3 A função write_line . . . . .                  | 13        |
| <b>4. Variáveis e valores</b>                      | <b>14</b> |
| 4.1 Alterando uma variável . . . . .               | 14        |
| 4.2 Constantes . . . . .                           | 14        |
| 4.3 Nomes recomendados . . . . .                   | 14        |
| 4.4 Inicialização definida . . . . .               | 14        |
| <b>5. Tipos básicos</b>                            | <b>15</b> |
| 5.1 Inteiros . . . . .                             | 15        |
| 5.2 Booleano . . . . .                             | 15        |
| 5.3 Ponto flutuante . . . . .                      | 15        |
| 5.4 Caractere, byte e texto . . . . .              | 15        |
| 5.5 void e never . . . . .                         | 16        |
| 5.6 Conversões explícitas . . . . .                | 16        |
| <b>PARTE II - Programação passo a passo</b>        | <b>17</b> |
| <b>6. Operações e expressões</b>                   | <b>17</b> |

|                                                         |           |
|---------------------------------------------------------|-----------|
| 6.1 Aritmética . . . . .                                | 17        |
| 6.2 Comparação . . . . .                                | 17        |
| 6.3 Lógica . . . . .                                    | 18        |
| 6.4 Bits . . . . .                                      | 18        |
| 6.5 Overflow . . . . .                                  | 18        |
| <b>7. Tomando decisões</b>                              | <b>18</b> |
| 7.1 if e else . . . . .                                 | 18        |
| 7.2 Várias possibilidades . . . . .                     | 18        |
| 7.3 switch . . . . .                                    | 19        |
| 7.4 Erro comum: usar atribuição como condição . . . . . | 19        |
| <b>8. Repetindo tarefas</b>                             | <b>19</b> |
| 8.1 while . . . . .                                     | 19        |
| 8.2 do while . . . . .                                  | 19        |
| 8.3 for . . . . .                                       | 19        |
| 8.4 loop . . . . .                                      | 20        |
| 8.5 break e continue . . . . .                          | 20        |
| <b>9. Criando funções</b>                               | <b>20</b> |
| 9.1 Partes da declaração . . . . .                      | 20        |
| 9.2 Chamando a função . . . . .                         | 21        |
| 9.3 Função void . . . . .                               | 21        |
| 9.4 Funções pequenas e focadas . . . . .                | 21        |
| <b>10. Arrays, fatias e texto</b>                       | <b>21</b> |
| 10.1 Array . . . . .                                    | 21        |
| 10.2 Verificação de limites . . . . .                   | 21        |
| 10.3 Fatia . . . . .                                    | 21        |
| 10.4 Texto e bytes . . . . .                            | 22        |
| <b>11. Estruturas e enums</b>                           | <b>22</b> |
| 11.1 Estrutura . . . . .                                | 22        |
| 11.2 Enum . . . . .                                     | 22        |
| 11.3 União . . . . .                                    | 22        |
| <b>12. Entrada de dados e erros</b>                     | <b>23</b> |
| 12.1 Por que existe ReadI32Result . . . . .             | 23        |
| 12.2 Erro esperado e panic . . . . .                    | 23        |
| <b>13. Trabalhando com arquivos</b>                     | <b>24</b> |
| 13.1 Abrir, usar e fechar . . . . .                     | 24        |
| 13.2 Por que defer é útil . . . . .                     | 24        |
| 13.3 Caminhos Linux não são sempre UTF-8 . . . . .      | 24        |
| <b>14. Pequenos projetos de aprendizagem</b>            | <b>25</b> |

|                                                       |           |
|-------------------------------------------------------|-----------|
| 14.1 Calculadora de código de saída . . . . .         | 25        |
| 14.2 Contador de elementos . . . . .                  | 25        |
| 14.3 Inspetor de manifesto . . . . .                  | 25        |
| <b>PARTE III - Design de software</b>                 | <b>27</b> |
| <b>15. Da ideia ao software</b>                       | <b>27</b> |
| 15.1 Comece pelo problema . . . . .                   | 27        |
| 15.2 Requisitos funcionais e não funcionais . . . . . | 27        |
| 15.3 Divida por responsabilidades . . . . .           | 28        |
| <b>16. Módulos, camadas e dependências</b>            | <b>28</b> |
| 16.1 Módulo . . . . .                                 | 28        |
| 16.2 Camadas do projeto . . . . .                     | 28        |
| 16.3 Dependência por interface . . . . .              | 29        |
| 16.4 Evite o “módulo de tudo” . . . . .               | 29        |
| <b>17. Modelando dados e estados</b>                  | <b>29</b> |
| 17.1 Use tipos que representem o domínio . . . . .    | 29        |
| 17.2 Estado explícito . . . . .                       | 29        |
| 17.3 Invariante . . . . .                             | 29        |
| <b>18. Projetando erros e observabilidade</b>         | <b>30</b> |
| 18.1 Erros precisam de contexto . . . . .             | 30        |
| 18.2 Não esconda a causa . . . . .                    | 30        |
| 18.3 Logs estruturados . . . . .                      | 30        |
| 18.4 Auditoria não é log comum . . . . .              | 30        |
| <b>19. Testando o design</b>                          | <b>31</b> |
| 19.1 Teste unitário . . . . .                         | 31        |
| 19.2 Teste de integração . . . . .                    | 31        |
| 19.3 Teste de aceitação . . . . .                     | 31        |
| 19.4 Teste de falha . . . . .                         | 31        |
| 19.5 Primeiro somente leitura . . . . .               | 31        |
| <b>20. Arquitetura do Bilarp Linux</b>                | <b>32</b> |
| 20.1 O Bilarp não substitui o Linux . . . . .         | 32        |
| 20.2 Fluxo de atualização . . . . .                   | 32        |
| 20.3 Controle do usuário . . . . .                    | 32        |
| 20.4 Não criar criptografia própria . . . . .         | 32        |
| 20.5 Não criar banco de dados próprio . . . . .       | 33        |
| <b>PARTE IV - Bibliotecas da C Larp</b>               | <b>34</b> |
| <b>21. Como as bibliotecas são organizadas</b>        | <b>34</b> |
| 21.1 Camadas . . . . .                                | 34        |
| 21.2 Regras gerais . . . . .                          | 34        |

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>22. Biblioteca core</b>                      | <b>35</b> |
| 22.1 core.types . . . . .                       | 35        |
| 22.2 core.numeric . . . . .                     | 35        |
| 22.3 core.memory . . . . .                      | 36        |
| 22.4 result e optional . . . . .                | 36        |
| <b>23. Biblioteca std</b>                       | <b>36</b> |
| 23.1 std.io . . . . .                           | 36        |
| 23.2 std.fs . . . . .                           | 37        |
| 23.3 std.path . . . . .                         | 37        |
| 23.4 std.process . . . . .                      | 37        |
| 23.5 std.time . . . . .                         | 37        |
| 23.6 std.log . . . . .                          | 37        |
| <b>24. Memória e coleções</b>                   | <b>38</b> |
| 24.1 Alocadores . . . . .                       | 38        |
| 24.2 Catálogo de coleções . . . . .             | 38        |
| 24.3 Escolha prática . . . . .                  | 38        |
| <b>25. Dados, sincronização e Linux</b>         | <b>39</b> |
| 25.1 Dados . . . . .                            | 39        |
| 25.2 Sincronização . . . . .                    | 39        |
| 25.3 Linux específico . . . . .                 | 39        |
| 25.4 Hardware . . . . .                         | 39        |
| <b>26. SDK do Bilarp</b>                        | <b>40</b> |
| 26.1 Manifesto . . . . .                        | 40        |
| 26.2 Plano . . . . .                            | 40        |
| 26.3 Transação . . . . .                        | 40        |
| 26.4 Índice de arquivos . . . . .               | 40        |
| 26.5 Política . . . . .                         | 41        |
| <b>PARTE V - Especificação técnica</b>          | <b>42</b> |
| <b>27. Arquivos, módulos e gramática léxica</b> | <b>42</b> |
| 27.1 Estrutura de arquivo . . . . .             | 42        |
| 27.2 Importação . . . . .                       | 42        |
| 27.3 Identificadores . . . . .                  | 42        |
| 27.4 Literais . . . . .                         | 43        |
| 27.5 Palavras reservadas . . . . .              | 43        |
| <b>28. Semântica numérica e avaliação</b>       | <b>43</b> |
| 28.1 Conversões . . . . .                       | 43        |
| 28.2 Ordem de avaliação . . . . .               | 43        |
| 28.3 Precedência . . . . .                      | 44        |

|                                                         |           |
|---------------------------------------------------------|-----------|
| 28.4 Falhas verificadas . . . . .                       | 44        |
| <b>29. Memória, ponteiros e unsafe</b>                  | <b>44</b> |
| 29.1 Armazenamento . . . . .                            | 44        |
| 29.2 Ponteiros . . . . .                                | 44        |
| 29.3 Operações unsafe . . . . .                         | 45        |
| 29.4 Limite da garantia . . . . .                       | 45        |
| 29.5 volatile e atômicos . . . . .                      | 45        |
| <b>30. Erros e limpeza técnica</b>                      | <b>45</b> |
| 30.1 Resultados concretos . . . . .                     | 45        |
| 30.2 Contrato de panic . . . . .                        | 45        |
| 30.3 defer . . . . .                                    | 46        |
| <b>31. FFI, ABI e atributos</b>                         | <b>46</b> |
| 31.1 Alvo inicial . . . . .                             | 46        |
| 31.2 Importar C . . . . .                               | 46        |
| 31.3 Exportar para C . . . . .                          | 46        |
| 31.4 Atributos . . . . .                                | 46        |
| 31.5 Compilação condicional . . . . .                   | 47        |
| 31.6 Assembly . . . . .                                 | 47        |
| <b>PARTE VI - Construindo o compilador e o produto</b>  | <b>48</b> |
| <b>32. Arquitetura do compilador</b>                    | <b>48</b> |
| 32.1 Bootstrap em Rust . . . . .                        | 48        |
| 32.2 Pipeline . . . . .                                 | 48        |
| 32.3 O que cada parte faz . . . . .                     | 49        |
| 32.4 Por que LLVM . . . . .                             | 49        |
| 32.5 Diagnóstico profissional . . . . .                 | 49        |
| <b>33. Ferramenta de terminal</b>                       | <b>50</b> |
| 33.1 Manifesto . . . . .                                | 50        |
| 33.2 Perfis . . . . .                                   | 50        |
| <b>34. Plano de implementação</b>                       | <b>51</b> |
| 34.1 Marco 0 - corpus oficial . . . . .                 | 51        |
| 34.2 Marco 1 - lexer . . . . .                          | 51        |
| 34.3 Marco 2 - parser . . . . .                         | 51        |
| 34.4 Marco 3 - semântica . . . . .                      | 51        |
| 34.5 Marco 4 - executável . . . . .                     | 51        |
| 34.6 Marco 5 - FFI e saída mínima . . . . .             | 51        |
| 34.7 Marco 6 - programas úteis . . . . .                | 51        |
| 34.8 Marco 7 - memória, genéricos e manifesto . . . . . | 51        |
| 34.9 Marco 8 - Bilarp somente leitura . . . . .         | 51        |
| 34.10 Marco 9 - planejamento . . . . .                  | 51        |

|                                               |           |
|-----------------------------------------------|-----------|
| 34.11 Marco 10 - transação . . . . .          | 52        |
| 34.12 Marco 11 - autocontenção . . . . .      | 52        |
| 34.13 Escala de tempo . . . . .               | 52        |
| <b>35. Como evitar um toy model</b>           | <b>52</b> |
| 35.1 Provas necessárias . . . . .             | 52        |
| 35.2 Programas de prova . . . . .             | 52        |
| 35.3 Testes do compilador . . . . .           | 53        |
| 35.4 Apresentação honesta . . . . .           | 53        |
| <b>36. Gramática orientativa</b>              | <b>53</b> |
| <b>37. Exercícios</b>                         | <b>55</b> |
| 37.1 Fundamentos . . . . .                    | 55        |
| 37.2 Decisões e laços . . . . .               | 55        |
| 37.3 Funções e estruturas . . . . .           | 55        |
| 37.4 Design . . . . .                         | 55        |
| 37.5 Engenharia . . . . .                     | 55        |
| <b>38. Referência rápida</b>                  | <b>56</b> |
| 38.1 Declarações . . . . .                    | 56        |
| 38.2 Funções . . . . .                        | 56        |
| 38.3 Controle . . . . .                       | 56        |
| 38.4 Tipos . . . . .                          | 56        |
| 38.5 Organização . . . . .                    | 56        |
| 38.6 Segurança delimitada . . . . .           | 56        |
| <b>39. Glossário</b>                          | <b>57</b> |
| <b>40. Checklist do projeto</b>               | <b>58</b> |
| Antes do lexer . . . . .                      | 58        |
| Antes da biblioteca . . . . .                 | 58        |
| Antes do Bilarp modificar o sistema . . . . . | 58        |
| <b>41. Conclusão</b>                          | <b>59</b> |



# Como usar esta apostila

Esta apostila foi escrita para dois públicos ao mesmo tempo. A primeira metade ensina os conceitos como se o leitor nunca tivesse usado C Larp. A segunda metade registra decisões de design, bibliotecas e arquitetura do compilador para quem participará da implementação.

Não é necessário conhecer Assembly, kernel ou compiladores para começar. É útil já ter visto alguma linguagem de programação, mas os conceitos essenciais são explicados no texto.

## Caminho de leitura

| Se você é...              | Comece por...   | Objetivo                                  |
|---------------------------|-----------------|-------------------------------------------|
| iniciante absoluto        | Partes I e II   | entender e escrever códigos simples       |
| estudante de programação  | Partes II e III | praticar lógica e organização de software |
| colaborador da biblioteca | Partes III e IV | entender módulos, tipos e contratos       |
| colaborador do compilador | Partes IV e V   | implementar a linguagem com precisão      |
| colaborador do Bilarp     | Partes III e VI | projetar ferramentas da distribuição      |

## Símbolos didáticos

| Marca          | Significado                    |
|----------------|--------------------------------|
| Conceito       | definição importante           |
| Exemplo        | código que demonstra uma ideia |
| Atenção        | erro comum ou limitação        |
| Engenharia     | detalhe para quem implementará |
| Estado E0 a E4 | maturidade do recurso          |

## Maturidade dos recursos

| Estado            | Significado                                 |
|-------------------|---------------------------------------------|
| E0 - definido     | existe apenas no documento                  |
| E1 - MVP          | necessário para o primeiro compilador real  |
| E2 - utilizável   | necessário para programas úteis             |
| E3 - Bilarp       | necessário para ferramentas da distribuição |
| E4 - experimental | pesquisa futura, sem promessa de prazo      |

# PARTE I - O projeto em linguagem simples

Uma nova etapa da apostila: avance no seu ritmo e consulte o glossário sempre que necessário.

## 1. O que estamos construindo

O projeto possui dois produtos diferentes:

- 1 **C Larp**: uma linguagem de programação compilada.
- 2 **Bilarp**: uma distribuição Linux baseada em uma distribuição já existente.

Esses produtos cooperam, mas não são a mesma coisa. A C Larp servirá, no futuro, para escrever ferramentas do Bilarp. O Bilarp continuará usando o kernel Linux e os componentes confiáveis da distribuição-base.

### 1.1 O que é uma linguagem de programação

Uma linguagem de programação é um conjunto de palavras, símbolos e regras usados para descrever tarefas para o computador. Assim como um idioma possui vocabulário e gramática, uma linguagem possui:

- palavras reservadas;
- tipos de valores;
- formas de tomar decisões;
- formas de repetir tarefas;
- formas de organizar códigos;
- regras que definem o significado de cada expressão.

Um arquivo C Larp terá extensão `.clarp`.

## 1.2 O que é um compilador

O computador não executa diretamente o texto escrito pelo programador. O compilador transforma esse texto em um programa nativo.

EXEMPLO: TEXTO

```
código C Larp
  |
  v
compilador clarp
  |
  v
código de máquina e executável Linux
```

O compilador também identifica erros. Se o programador tentar somar texto com um tipo incompatível, usar uma variável inexistente ou esquecer um retorno obrigatório, o compilador deve explicar o problema.

## 1.3 O que é uma distribuição Linux

Linux é o kernel que controla processos, memória, dispositivos e outros recursos centrais. Uma distribuição reúne o kernel com bibliotecas, instalador, programas, repositórios e políticas.

O Bilarp será baseado em outra distribuição. Isso significa que não criaremos um kernel nem reconstruiremos todo o ecossistema. Acrescentaremos uma camada voltada a:

- inspeção de arquivos e pacotes;
- planejamento de atualizações;
- políticas definidas pelo usuário;
- histórico auditável;
- snapshots e rollback quando suportados;
- integração controlada com o gerenciador de pacotes existente.

## 1.4 O que não vamos prometer

- A linguagem ainda não está pronta.
- A C Larp não é “mais rápida que C” sem medições reais.
- `unsafe` não torna todo o restante automaticamente seguro.
- Um programa comum não ganha controle irrestrito do hardware.
- O Bilarp não terá kernel próprio neste projeto.
- Não construiremos compilador, distribuição, banco de dados e todos os módulos ao mesmo tempo.

**Conceito:** um projeto real não é aquele que promete tudo. É aquele que entrega uma parte pequena, testável e verdadeira antes de crescer.

## 2. Conhecendo um código C Larp

Observe o menor programa planejado:

EXEMPLO: C LARP

```
namespace examples.first;

bi0 main() {
    finish to 0;
}
```

### 2.1 Explicação linha por linha

`namespace examples.first;`

Define o endereço lógico do código. Um namespace organiza nomes e evita que duas partes diferentes do programa usem o mesmo nome por acidente. Os pontos separam níveis de organização.

`bi0 main()`

Declara a entrada de um programa comum. Quando o sistema operacional inicia o executável, é essa função que começa o trabalho. `bi0` informa que se trata da entrada hospedada da C Larp.

`{ e }`

As chaves delimitam um bloco. Tudo entre elas pertence à função.

`finish to 0;`

Encerra a função e devolve o código zero ao Linux. Por convenção, zero significa sucesso. O ponto e vírgula encerra a instrução.

### 2.2 Espaços e indentação

O compilador distingue símbolos e palavras, mas a indentação ajuda pessoas a compreender o código:

EXEMPLO: C LARP

```
bi0 main() {
    finish to 0;
}
```

Esta forma é preferida a:

EXEMPLO: C LARP

```
bi0 main(){finish to 0;}
```

O formatador `clarp fmt` será responsável por aplicar o estilo oficial.

### 2.3 Comentários

Comentários explicam decisões e são ignorados pelo compilador:

EXEMPLO: C LARP

```
// Comentário de uma linha

/* Comentário que pode
   ocupar mais de uma linha */

/// Documentação pública de uma função.
```

Um comentário bom explica **por que** algo foi feito. Repetir literalmente a instrução costuma acrescentar pouco.

## 3. Mostrando mensagens

Entrada e saída pertencem à biblioteca, não à gramática da linguagem. Para escrever no terminal, importamos `std.io`.

EXEMPLO: C LARP

```
namespace examples.hello;

use<std.io> as io;

bi0 main() {
    io::write_line("Olá, Bilarp!");
    finish to 0;
}
```

### 3.1 O que significa `use`

`use<std.io> as io;` pede acesso ao módulo `std.io` e cria o apelido `io`. Depois usamos `io::` para indicar que uma função pertence a esse módulo.

Esse formato é mais claro que importar todos os nomes para o mesmo espaço.

### 3.2 Texto entre aspas

`"Olá, Bilarp!"` é um literal de texto. O tipo planejado é `str`, uma visão imutável de bytes UTF-8 válidos.

### 3.3 A função `write_line`

`io::write_line` envia texto para a saída e acrescenta uma nova linha. Na biblioteca real, a operação devolverá um resultado porque a escrita pode falhar.

Uma versão mais cuidadosa será:

EXEMPLO: C LARP

```
namespace examples.checked_output;

use<std.io> as io;

bi0 main() {
    get io::Status as status = io::write_line("Olá!");

    if (!status.ok) {
        finish to status.code;
    }

    finish to 0;
}
```

Os nomes `bout1` e `bin0`, imaginados no início do projeto, poderão existir como funções de conveniência dentro de `std.io`. Eles não serão palavras reservadas. Isso permite que a linguagem exista mesmo em programas que não usam terminal.

**Atenção:** os exemplos de I/O são E2. O primeiro compilador E1 começará apenas retornando códigos ao Linux.

## 4. Variáveis e valores

Uma variável é um nome associado a um espaço que guarda um valor.

EXEMPLO: C LARP

```
get i32 as age = 18;
```

Esta declaração possui quatro partes:

| Parte               | Significado                                 |
|---------------------|---------------------------------------------|
| <code>get</code>    | inicia a declaração de uma variável mutável |
| <code>i32</code>    | tipo inteiro de 32 bits com sinal           |
| <code>as age</code> | escolhe o nome <code>age</code>             |
| <code>= 18</code>   | fornece o valor inicial                     |

### 4.1 Alterando uma variável

EXEMPLO: C LARP

```
get i32 as score = 10;  
score = 25;  
score += 5;
```

Depois dessas instruções, `score` vale 30. A operação `+=` soma e grava o resultado na mesma variável.

### 4.2 Constantes

Uma constante não muda:

EXEMPLO: C LARP

```
const i32 MAX_ATTEMPTS = 3;
```

Constantes são úteis para nomes que expressam significado. `MAX_ATTEMPTS` explica melhor a regra que repetir o número 3 em vários lugares.

### 4.3 Nomes recomendados

EXEMPLO: C LARP

```
get i32 as package_count = 0;  
get bool as update_available = false;  
const usize BUFFER_SIZE = 4096;
```

- Variáveis e funções usam `snake_case`.
- Tipos usam `PascalCase`.
- Constantes globais usam `UPPER_SNAKE_CASE`.
- Nomes devem descrever propósito, não apenas o tipo.

### 4.4 Inicialização definida

O compilador não permitirá ler uma variável que talvez não tenha recebido valor:

EXEMPLO: C LARP

```
get i32 as code;

if (condition) {
    code = 10;
} else {
    code = 20;
}

finish to code;
```

Como os dois caminhos atribuem um valor, a leitura final é válida.

## 5. Tipos básicos

Um tipo informa quais valores são possíveis, quanto espaço eles ocupam e quais operações fazem sentido.

### 5.1 Inteiros

| Tipo                                 | Descrição                                    | Exemplo de uso              |
|--------------------------------------|----------------------------------------------|-----------------------------|
| <code>i8, i16, i32, i64, i128</code> | inteiros com sinal                           | temperatura, saldo, código  |
| <code>u8, u16, u32, u64, u128</code> | inteiros sem sinal                           | bytes, máscaras, contadores |
| <code>usize</code>                   | tamanho e índice da plataforma               | comprimento de array        |
| <code>isize</code>                   | diferença do tamanho da plataforma           | deslocamentos               |
| <code>byte</code>                    | apelido de <code>u8</code> para dados brutos | buffers e arquivos          |

O número em `i32` indica a quantidade de bits. Com sinal significa que o valor pode ser negativo.

### 5.2 Booleano

`bool` possui somente `true` e `false`:

EXEMPLO: C LARP

```
get bool as is_ready = true;
```

Condições aceitam `bool`. A C Larp não transforma automaticamente zero em falso e outros números em verdadeiro.

### 5.3 Ponto flutuante

`f32` e `f64` representam valores com parte fracionária:

EXEMPLO: C LARP

```
get f64 as percentage = 82.5f64;
```

Valores de ponto flutuante não representam toda fração decimal exatamente. Dinheiro e versões de pacote não devem depender de comparação ingênua com `f64`.

### 5.4 Caractere, byte e texto

EXEMPLO: C LARP

```
get char as letter = 'A';
get u8 as ascii_byte = b'A';
get str as message = "Olá";
```

- `char` representa um valor Unicode válido.

- `b'A'` é um byte.
- `str` representa texto UTF-8 imutável.

## 5.5 `void` e `never`

- `void` indica que uma função não devolve valor.
- `never` indica que a função não retorna ao chamador.

## 5.6 Conversões explícitas

Tipos numéricos diferentes não são misturados silenciosamente:

EXEMPLO: C LARP

```
get u32 as small = 10u32;  
get u64 as large = cast<u64>(small);
```

Exigir `cast` evita que perda de informação fique escondida.



# PARTE II - Programação passo a passo

Uma nova etapa da apostila: avance no seu ritmo e consulte o glossário sempre que necessário.

## 6. Operações e expressões

Uma expressão produz um valor. `2 + 3` produz 5; `age >= 18` produz um booleano.

### 6.1 Aritmética

EXEMPLO: C LARP

```
get i32 as a = 10;
get i32 as b = 3;

get i32 as sum = a + b;
get i32 as difference = a - b;
get i32 as product = a * b;
get i32 as quotient = a / b;
get i32 as remainder = a % b;
```

% devolve o resto da divisão inteira. Divisão por zero é uma falha controlada.

### 6.2 Comparação

| Operador                  | Pergunta               |
|---------------------------|------------------------|
| <code>==</code>           | os valores são iguais? |
| <code>!=</code>           | são diferentes?        |
| <code>&lt; e &lt;=</code> | o primeiro é menor?    |
| <code>&gt; e &gt;=</code> | o primeiro é maior?    |

EXEMPLO: C LARP

```
get bool as adult = age >= 18;
```

## 6.3 Lógica

EXEMPLO: C LARP

```
get bool as can_update = is_admin && network_ready;
get bool as should_warn = has_risk || disk_is_full;
get bool as unavailable = !available;
```

- `&&` exige que as duas condições sejam verdadeiras.
- `||` exige pelo menos uma.
- `!` inverte o booleano.

## 6.4 Bits

EXEMPLO: C LARP

```
get u32 as flags = 0u32;
flags = flags | 0b0001u32;
get bool as first_bit = (flags & 0b0001u32) != 0u32;
```

Operações binárias são úteis para permissões, formatos e protocolos. Iniciantes não precisam dominá-las para escrever programas comuns.

## 6.5 Overflow

Quando uma soma ultrapassa o limite do tipo, o operador normal chama `panic`. Para comportamentos intencionais, `core.numeric` oferecerá:

- `checked_add` para detectar overflow;
- `wrap_add` para descartar bits excedentes;
- `saturating_add` para permanecer no limite.

O comportamento será igual em modo de desenvolvimento e otimizado.

# 7. Tomando decisões

## 7.1 if e else

EXEMPLO: C LARP

```
if (age >= 18) {
    io::write_line("Maior de idade");
} else {
    io::write_line("Menor de idade");
}
```

O programa avalia a condição. Se for verdadeira, executa o primeiro bloco. Caso contrário, executa o bloco `else`.

## 7.2 Várias possibilidades

EXEMPLO: C LARP

```
if (temperature >= 80) {
    io::write_line("Alerta");
} else if (temperature >= 60) {
    io::write_line("Atenção");
} else {
    io::write_line("Normal");
}
```

As condições são avaliadas na ordem. O primeiro bloco verdadeiro é executado.

## 7.3 switch

`switch` é adequado quando uma expressão pode assumir vários valores discretos:

EXEMPLO: C LARP

```
switch (command) {
    case 1:
        inspect();

    case 2:
        plan_update();

    case 3:
        show_history();

    default:
        show_help();
}
```

Um `case` não cai automaticamente no próximo. Para permitir queda intencional, o programador escreve `fallthrough`.

## 7.4 Erro comum: usar atribuição como condição

EXEMPLO: C LARP

```
if (value = 10) { // erro
}
```

Atribuição não produz valor na C Larp. A comparação correta é:

EXEMPLO: C LARP

```
if (value == 10) {
}
```

# 8. Repetindo tarefas

## 8.1 while

EXEMPLO: C LARP

```
get i32 as count = 0;

while (count < 3) {
    io::write_i32(count);
    count += 1;
}
```

Antes de cada repetição, `while` verifica a condição.

## 8.2 do while

EXEMPLO: C LARP

```
do {
    try_again();
} while (failed);
```

O bloco executa pelo menos uma vez e a condição é verificada no final.

## 8.3 for

EXEMPLO: C LARP

```
for (get usize as i = 0; i < 10; i += 1) {
    io::write_usize(i);
}
```

O `for` possui:

- 1 inicialização: `get usize as i = 0;`
- 2 condição: `i < 10;`
- 3 atualização: `i += 1.`

## 8.4 loop

EXEMPLO: C LARP

```
loop {
    process_next_event();
}
```

`loop` cria repetição sem condição. Deve existir uma saída interna ou a função nunca terminará.

## 8.5 break e continue

EXEMPLO: C LARP

```
for (get usize as i = 0; i < items.length; i += 1) {
    if (items[i].ignored) {
        continue;
    }

    if (items[i].fatal) {
        break;
    }

    process(items[i]);
}
```

`continue` pula para a próxima repetição. `break` encerra o laço.

# 9. Criando funções

Uma função reúne uma tarefa que pode ser chamada por outras partes do programa.

EXEMPLO: C LARP

```
fn i32 sum(get i32 as a, get i32 as b) {
    finish to a + b;
}
```

## 9.1 Partes da declaração

| Parte                  | Significado             |
|------------------------|-------------------------|
| <code>fn</code>        | inicia uma função comum |
| <code>i32</code>       | tipo devolvido          |
| <code>sum</code>       | nome da função          |
| parâmetros             | valores recebidos       |
| bloco                  | instruções executadas   |
| <code>finish to</code> | valor devolvido         |

## 9.2 Chamando a função

EXEMPLO: C LARP

```
get i32 as result = sum(20, 22);
```

Os argumentos são avaliados da esquerda para a direita.

## 9.3 Função `void`

EXEMPLO: C LARP

```
fn void show_separator() {  
    io::write_line("-----");  
    finish;  
}
```

`void` significa que não existe valor de retorno. `finish`; apenas encerra a função.

## 9.4 Funções pequenas e focadas

Uma função deve ter uma responsabilidade clara. Em vez de uma função que lê arquivos, decide políticas, aplica atualizações e grava logs, prefira funções separadas:

EXEMPLO: TEXTO

```
read_manifest  
validate_manifest  
build_plan  
evaluate_policy  
apply_transaction  
write_audit_event
```

Essa divisão simplifica testes e revisão.

# 10. Arrays, fatias e texto

## 10.1 Array

Um array contém uma quantidade fixa de elementos:

EXEMPLO: C LARP

```
get [4]i32 as values = [10, 20, 30, 40];  
get i32 as first = values[0];
```

O primeiro índice é zero. Neste exemplo, os índices válidos vão de 0 a 3.

## 10.2 Verificação de limites

EXEMPLO: C LARP

```
get i32 as invalid = values[10];
```

O acesso é inválido. Quando o índice só for conhecido durante a execução, a C Larp verificará o limite e chamará `panic` se estiver fora dele.

## 10.3 Fatia

Uma fatia observa parte de um armazenamento existente. Ela contém endereço e comprimento, mas não é proprietária da memória.

EXEMPLO: C LARP

```
fn i32 first_value(get []i32 as values) {
    if (values.length == 0) {
        finish to 0;
    }

    finish to values[0];
}
```

Uma fatia pode ficar inválida se o armazenamento original terminar. A primeira versão da linguagem não terá análise automática de lifetime.

## 10.4 Texto e bytes

`str` representa texto UTF-8 válido. `[]u8` representa bytes e pode conter qualquer dado.

EXEMPLO: C LARP

```
get str as title = "Bilarp";
get [256]u8 as raw_buffer;
```

Converter bytes recebidos de arquivo ou rede para `str` exige validação com `data.utf8`.

# 11. Estruturas e enums

## 11.1 Estrutura

Uma estrutura reúne campos relacionados:

EXEMPLO: C LARP

```
struct Package {
    get str as name;
    get str as version;
    get bool as installed;
}

get Package as editor = Package {
    name: "editor",
    version: "1.0.0",
    installed: true
};
```

Em vez de manter três variáveis soltas, usamos um valor que representa um pacote.

## 11.2 Enum

Um enum representa um conjunto fechado de opções:

EXEMPLO: C LARP

```
enum UpdateState : u8 {
    discovered = 0,
    planned = 1,
    approved = 2,
    applied = 3,
    failed = 4
}
```

O tipo `u8` determina a representação numérica. Isso é importante em arquivos e FFI.

## 11.3 União

Uma `union` permite diferentes interpretações da mesma memória. É um recurso avançado:

EXEMPLO: C LARP

```
union NumericValue {
    get i64 as integer;
    get f64 as decimal;
}
```

Ler um campo diferente do último campo escrito exige `unsafe`. Em programas comuns, uma estrutura com enum de estado costuma ser mais clara.

## 12. Entrada de dados e erros

Ler do terminal pode falhar: o usuário pode digitar texto quando esperamos um número, encerrar a entrada ou produzir dados maiores que o buffer.

EXEMPLO: C LARP

```
namespace examples.read_number;

use<std.io> as io;

bi0 main() {
    get io::ReadI32Result as input = io::read_i32();

    if (!input.ok) {
        io::write_error("Número inválido");
        finish to input.code;
    }

    io::write_i32(input.value);
    finish to 0;
}
```

### 12.1 Por que existe `ReadI32Result`

A função precisa devolver duas informações:

- se a leitura funcionou;
- o valor ou o código do erro.

Uma estrutura concreta atende a primeira biblioteca. Depois de genéricos, a linguagem usará `result<T,E>`.

### 12.2 Erro esperado e `panic`

Erro esperado é algo que o programa consegue tratar, como arquivo inexistente ou entrada inválida. `panic` é reservado para invariantes quebradas, acesso fora de limite, overflow normal ou divisão por zero.

`panic` encerra o processo, não retorna e não executa `defer` durante desenrolamento, pois não haverá desenrolamento de pilha.

## 13. Trabalhando com arquivos

EXEMPLO: C LARP

```
namespace examples.file_size;

use<std.fs> as fs;

bi0 main() {
    get fs::OpenResult as opened = fs::open(
        "/etc/os-release",
        fs::READ_ONLY
    );

    if (!opened.ok) {
        finish to opened.code;
    }

    defer fs::close(opened.file);

    get fs::MetadataResult as metadata = fs::metadata(opened.file);

    if (!metadata.ok) {
        finish to metadata.code;
    }

    finish to 0;
}
```

### 13.1 Abrir, usar e fechar

Um arquivo é um recurso do sistema operacional. `fs::open` tenta abri-lo. Se funcionar, `defer fs::close` garante fechamento nas saídas normais do bloco.

### 13.2 Por que `defer` é útil

Sem `defer`, cada caminho de retorno precisaria lembrar de fechar o arquivo. O comando agenda a limpeza para a saída normal do escopo, em ordem inversa.

### 13.3 Caminhos Linux não são sempre UTF-8

Nomes de arquivo no Linux podem conter bytes que não formam texto UTF-8. Por isso, `std.path` terá um tipo de caminho separado de `str`. Interfaces para pessoas podem produzir uma versão exibível, mas não devem perder os bytes originais silenciosamente.



# 14. Pequenos projetos de aprendizagem

## 14.1 Calculadora de código de saída

Objetivos:

- declarar variáveis;
- usar operadores;
- criar função;
- devolver o resultado ao Linux.

EXEMPLO: C LARP

```
namespace practice.exit_sum;

fn i32 add(get i32 as a, get i32 as b) {
    finish to a + b;
}

bi0 main() {
    get i32 as result = add(10, 20);
    finish to result;
}
```

## 14.2 Contador de elementos

Objetivos:

- usar array;
- percorrer com `for`;
- trabalhar com `usize`.

EXEMPLO: C LARP

```
namespace practice.count;

bi0 main() {
    get [4]i32 as values = [4, 8, 15, 16];
    get usize as count = 0;

    for (get usize as i = 0; i < values.length; i += 1) {
        count += 1;
    }

    finish to cast<i32>(count);
}
```

## 14.3 Inspetor de manifesto

Este projeto será feito mais tarde. Ele deve:

- 1 receber o caminho de um manifesto;
- 2 abrir o arquivo;
- 3 limitar o tamanho lido;
- 4 interpretar campos sem executar comandos;
- 5 validar versão e estrutura;
- 6 mostrar erros com linha e coluna;
- 7 não modificar o sistema.

Esse será um bom primeiro programa real da C Larp porque exercita arquivos, texto, estruturas, erros e testes sem exigir privilégios administrativos.

## PARTE III - Design de software

Uma nova etapa da apostila: avance no seu ritmo e consulte o glossário sempre que necessário.

### 15. Da ideia ao software

Programar não é apenas escrever instruções. Design de software é decidir como o problema será dividido, quais dados existirão, quais erros podem ocorrer e como as partes conversarão.

#### 15.1 Comece pelo problema

Antes de escolher estruturas ou bibliotecas, escreva:

- quem usará o programa;
- qual tarefa deseja concluir;
- quais entradas serão fornecidas;
- quais saídas serão produzidas;
- o que não pode acontecer;
- como uma falha será percebida e recuperada.

Exemplo para um inspetor de pacotes:

“Como administrador, quero informar o nome de um pacote e descobrir sua versão, seus arquivos e sua origem, sem alterar o sistema.”

Essa frase já indica que a primeira ferramenta deve ser somente leitura.

#### 15.2 Requisitos funcionais e não funcionais

Requisito funcional descreve uma ação: listar arquivos de um pacote. Requisito não funcional descreve uma qualidade: concluir sem modificar o sistema, registrar erros e responder em tempo aceitável.

| Tipo           | Exemplo                                    |
|----------------|--------------------------------------------|
| funcional      | consultar pacote por nome                  |
| funcional      | mostrar versão instalada                   |
| segurança      | não executar entrada do usuário como shell |
| confiabilidade | não alterar arquivos em modo de inspeção   |
| desempenho     | limitar leitura de manifestos              |
| usabilidade    | explicar pacote não encontrado             |

## 15.3 Dívida por responsabilidades

EXEMPLO: TEXTO

```
CLI
|
v
serviço de inspeção
|
+--> adaptador da distribuição-base
+--> índice de arquivos
+--> formatação da resposta
```

A CLI entende argumentos e mostra mensagens. O serviço executa a regra de negócio. O adaptador conversa com a distribuição-base. Essa separação permite testar cada parte.

# 16. Módulos, camadas e dependências

## 16.1 Módulo

Um módulo agrupa tipos e funções relacionados. `std.fs` cuida de arquivos; `bilarp.policy` cuida de regras do usuário.

Um bom módulo possui:

- nome específico;
- interface pública pequena;
- detalhes privados;
- erros documentados;
- dependências conhecidas;
- testes do contrato.

## 16.2 Camadas do projeto

EXEMPLO: TEXTO

```
Interface: CLI e mensagens
|
Aplicação: casos de uso e coordenação
|
Domínio: pacote, plano, política, transação
|
Infraestrutura: arquivos, SQLite, processos, rede
|
Linux e distribuição-base
```

Camadas superiores podem depender de contratos inferiores. O domínio não deve conhecer detalhes de cores do terminal ou comandos específicos do gerenciador da base.

## 16.3 Dependência por interface

O planejador não precisa saber se os dados vêm de um comando externo, SQLite ou arquivo. Ele precisa de uma interface capaz de consultar pacotes.

EXEMPLO: TEXTO

```
PackageSource
- find_by_name
- list_installed
- list_updates
```

O adaptador da distribuição-base implementa esse contrato. Em testes, uma implementação falsa fornece dados previsíveis.

## 16.4 Evite o “módulo de tudo”

Um módulo chamado `system` que reúne arquivo, rede, atualização, hardware e coleções esconde dependências. Prefira módulos menores e nomes claros.

# 17. Modelando dados e estados

## 17.1 Use tipos que representem o domínio

EXEMPLO: C LARP

```
struct PackageId {
    get str as name;
    get str as version;
    get str as architecture;
}
```

Uma estrutura evita passar três textos sem saber qual é qual.

## 17.2 Estado explícito

EXEMPLO: C LARP

```
enum TransactionState : u8 {
    created = 0,
    prepared = 1,
    applying = 2,
    verifying = 3,
    committed = 4,
    failed = 5,
    reverted = 6
}
```

Uma transação não deve saltar de `created` para `committed`. As transições válidas fazem parte do design e precisam de testes.

## 17.3 Invariante

Invariante é uma condição que sempre precisa ser verdadeira. Exemplos:

- um plano aprovado não muda;
- um artefato aplicado possui hash verificado;
- um arquivo gerenciado possui proprietário conhecido;
- uma transação confirmada possui evento de auditoria;
- uma política bloqueada não é ignorada silenciosamente.

Invariantes quebradas podem justificar `panic` durante desenvolvimento. Entradas inválidas do usuário retornam erros normais.

## 18. Projetando erros e observabilidade

### 18.1 Erros precisam de contexto

“Falhou” não é suficiente. Um erro útil informa:

- qual operação falhou;
- qual recurso estava envolvido;
- o código da causa;
- se é possível tentar novamente;
- qual ação o usuário pode tomar.

### 18.2 Não esconda a causa

EXEMPLO: TEXTO

```
erro: não foi possível abrir o manifesto
caminho: /var/lib/bilarp/repo/main.manifest
causa: permissão negada
ação: execute com a permissão apropriada ou verifique o proprietário
```

### 18.3 Logs estruturados

Um evento de log deve conter horário, nível, componente, tipo do evento e campos. Mensagens de segurança nunca registram senhas, tokens ou chaves privadas.

### 18.4 Auditoria não é log comum

Log ajuda a diagnosticar. Auditoria registra decisões e alterações relevantes. Um evento de auditoria pode conter:

- usuário ou processo responsável;
- plano aprovado;
- política aplicada;
- pacotes afetados;
- resultado e reversão;
- versão das ferramentas.

# 19. Testando o design

---

## 19.1 Teste unitário

Verifica uma função ou tipo isolado. Exemplo: uma regra de política deve bloquear downgrade.

## 19.2 Teste de integração

Verifica módulos juntos. Exemplo: o adaptador lê dados da distribuição-base e produz `PackageId` correto.

## 19.3 Teste de aceitação

Verifica o resultado para o usuário. Exemplo: `bilarp inspect editor` mostra versão e arquivos sem modificar o sistema.

## 19.4 Teste de falha

Sistemas confiáveis são testados também quando algo dá errado:

- arquivo incompleto;
- disco cheio;
- processo interrompido;
- assinatura inválida;
- rede encerrada;
- snapshot indisponível;
- estado alterado depois do planejamento.

## 19.5 Primeiro somente leitura

A ordem profissional para o Bilarp é:

- 1 inspecionar;
- 2 explicar;
- 3 simular;
- 4 planejar;
- 5 testar em máquina virtual;
- 6 aplicar com journal;
- 7 verificar;
- 8 oferecer reversão quando real.

## 20. Arquitetura do Bilarp Linux

### 20.1 O Bilarp não substitui o Linux

O Bilarp usa:

- kernel Linux;
- sistema de arquivos e serviços existentes;
- repositórios e pacotes da distribuição-base;
- gerenciador de pacotes da base na primeira versão.

O valor do Bilarp estará na camada de controle e explicação.

### 20.2 Fluxo de atualização

EXEMPLO: TEXTO

```
descobrir
|
resolver dependências
|
produzir plano
|
avaliar políticas
|
mostrar ao usuário
|
aprovar
|
snapshot quando disponível
|
aplicar e registrar journal
|
verificar
|
confirmar ou reverter
```

### 20.3 Controle do usuário

O administrador poderá:

- fazer dry-run;
- aprovar ou adiar;
- fixar versão;
- bloquear downgrade;
- escolher canal;
- preservar arquivo local e decidir conflito;
- consultar histórico;
- exigir snapshot para mudanças de risco;
- reverter quando o backend suportar.

Controle não significa eliminar assinatura ou integridade. A ferramenta alerta quando uma decisão reduz a segurança.

### 20.4 Não criar criptografia própria

Assinaturas e hashes criptográficos devem usar bibliotecas auditadas. A C Larp e o Bilarp apenas oferecem wrappers, política de confiança e registro da decisão.



## 20.5 Não criar banco de dados próprio

O índice de arquivos poderá usar SQLite por FFI ou outro armazenamento testado. Criar um banco de dados transacional seria outro projeto e não aumenta a credibilidade da primeira distribuição.

## PARTE IV - Bibliotecas da C Larp

Uma nova etapa da apostila: avance no seu ritmo e consulte o glossário sempre que necessário.

## 21. Como as bibliotecas são organizadas

A linguagem central conhece tipos, expressões e regras. Bibliotecas fornecem tarefas como abrir arquivo, mostrar mensagem e organizar coleções.

### 21.1 Camadas

| Prefixo                   | Conteúdo                       | Exemplo                         |
|---------------------------|--------------------------------|---------------------------------|
| <code>core</code>         | base conhecida pelo compilador | números, layout e panic         |
| <code>std</code>          | programas comuns no Linux      | I/O, arquivos e processos       |
| <code>memory</code>       | alocadores e armazenamento     | arena e mapping                 |
| <code>collections</code>  | estruturas de dados            | vector e hash_map               |
| <code>data</code>         | bytes, UTF-8 e serialização    | endian e validação              |
| <code>sync</code>         | concorrência                   | atomic e mutex                  |
| <code>linux</code>        | recursos específicos do Linux  | mmap e netlink                  |
| <code>bilarp</code>       | domínio da distribuição        | plano e transação               |
| <code>experimental</code> | pesquisas instáveis            | Assembly e dispositivo avançado |

### 21.2 Regras gerais

- Alocação é explícita e documentada.
- Bloqueio é documentado.
- Erro esperado retorna valor.

- `panic` não substitui tratamento de erro.
- Recurso aberto possui encerramento explícito.
- Wrappers públicos reduzem regiões `unsafe`.
- Biblioteca planejada não é anunciada como implementada.

## 22. Biblioteca `core`

`core` é importada implicitamente. Ela não depende de terminal, arquivo ou rede.

| Módulo                     | Estado | O que oferece                  | Quando usar           | Por que serve              |
|----------------------------|--------|--------------------------------|-----------------------|----------------------------|
| <code>core.types</code>    | E1     | tipos, tamanho e alinhamento   | sempre                | base comum da linguagem    |
| <code>core.numeric</code>  | E1     | aritmética alternativa e bits  | cálculos controlados  | evitar overflow ambíguo    |
| <code>core.panic</code>    | E1     | panic, trap e assert           | invariantes quebradas | falha uniforme             |
| <code>core.memory</code>   | E2     | copy, move, fill e compare     | buffers               | operações explícitas       |
| <code>core.compare</code>  | E2     | contratos de comparação        | ordenação             | comportamento reutilizável |
| <code>core.result</code>   | E2     | <code>result&lt;T,E&gt;</code> | erros esperados       | substituir exceções        |
| <code>core.optional</code> | E2     | <code>optional&lt;T&gt;</code> | valor que pode faltar | evitar sentinela mágica    |

### 22.1 `core.types`

Funções conhecidas em compilação:

EXEMPLO: C LARP

```
size_of<Package>()
align_of<Package>()
offset_of<Package>(installed)
```

Elas ajudam FFI, formatos binários e diagnóstico de layout.

### 22.2 `core.numeric`

EXEMPLO: TEXTO

```
checked_add, checked_sub, checked_mul
wrap_add, wrap_sub, wrap_mul
saturating_add, saturating_sub
rotate_left, rotate_right
count_ones, count_zeros
leading_zeros, trailing_zeros
byte_swap
```

Use `wrap_*` apenas quando o algoritmo exigir comportamento modular. Operações binárias antes atribuídas a `hardware.binary.logic` foram divididas entre `core.numeric` e `data.endian`, pois trabalhar com bits não significa acessar hardware.

## 22.3 `core.memory`

| Função               | Ação                            | Versão verificada         |
|----------------------|---------------------------------|---------------------------|
| <code>copy</code>    | copia regiões sem sobreposição  | usa fatias e comprimentos |
| <code>move</code>    | move regiões que podem sobrepor | usa fatias                |
| <code>fill</code>    | preenche bytes                  | verifica destino          |
| <code>compare</code> | compara bytes                   | não interpreta texto      |
| <code>swap</code>    | troca valores                   | depende do tipo           |

Versões com ponteiros brutos exigem `unsafe`.

## 22.4 `result` e `optional`

Depois de genéricos:

EXEMPLO: C LARP

```
result<File, FsError>
optional<PackageId>
```

Antes disso, cada módulo usa tipos concretos como `OpenResult`.

# 23. Biblioteca `std`

`std` é voltada a programas Linux.

| Módulo                           | Estado | Tipos principais                                                       | Funcionalidades                                                | Uso típico        |
|----------------------------------|--------|------------------------------------------------------------------------|----------------------------------------------------------------|-------------------|
| <code>std.io</code>              | E2     | <code>Status</code> , <code>ReadResult</code> , <code>Writer</code>    | <code>stdin</code> , <code>stdout</code> e <code>stderr</code> | CLI               |
| <code>std.fs</code>              | E2     | <code>File</code> , <code>Metadata</code> , <code>OpenResult</code>    | arquivo e diretório                                            | manifestos        |
| <code>std.path</code>            | E2     | <code>Path</code> , <code>PathBuffer</code>                            | combinar e consultar caminhos                                  | arquivos Linux    |
| <code>std.process</code>         | E2     | <code>Command</code> , <code>Process</code> , <code>ExitStatus</code>  | iniciar processo                                               | adaptador da base |
| <code>std.env</code>             | E2     | <code>ArgList</code> , <code>Environment</code>                        | argumentos e ambiente                                          | configuração      |
| <code>std.time</code>            | E2     | <code>Instant</code> , <code>Duration</code> , <code>SystemTime</code> | tempo e timeout                                                | auditoria         |
| <code>std.log</code>             | E2     | <code>Logger</code> , <code>Level</code> , <code>Record</code>         | logs estruturados                                              | diagnóstico       |
| <code>std.net</code>             | E3     | <code>Socket</code> , <code>Address</code> , <code>TlsClient</code>    | rede                                                           | repositório       |
| <code>std.dynamic_library</code> | E3     | <code>Library</code> , <code>Symbol</code>                             | biblioteca dinâmica                                            | plugins e FFI     |

## 23.1 `std.io`

Funções iniciais:

EXEMPLO: TEXTO

```
write(text) -> Status
write_line(text) -> Status
write_i32(value) -> Status
write_error(text) -> Status
read_line(buffer) -> ReadResult
read_i32() -> ReadI32Result
flush_stdout() -> Status
```

`read_line` recebe um buffer para não esconder alocação. `read_i32` é uma conveniência tipada. Futuramente `io::bout1` poderá formatar vários valores; `io::bin0_i32` poderá manter o nome histórico de entrada, sempre devolvendo erro explícito.

## 23.2 `std.fs`

| Tipo                     | O que representa                         |
|--------------------------|------------------------------------------|
| <code>File</code>        | arquivo aberto                           |
| <code>OpenMode</code>    | leitura, escrita, criação ou truncamento |
| <code>Metadata</code>    | tamanho, tipo, permissões e tempos       |
| <code>OpenResult</code>  | arquivo ou erro                          |
| <code>ReadResult</code>  | quantidade lida ou erro                  |
| <code>WriteResult</code> | quantidade escrita ou erro               |

Funções: `open`, `close`, `read`, `write`, `seek`, `sync`, `metadata`, `rename`, `remove_file`, `create_dir`, `read_link` e `list_dir`.

Operações destrutivas validam caminhos. Substituições preferem `rename` atômico quando o sistema de arquivos permitir. Durabilidade usa `sync` quando o contrato exigir.

## 23.3 `std.path`

`Path` observa um caminho e `PathBuffer` possui armazenamento. O módulo junta componentes, encontra o pai, consulta extensão e mantém bytes não UTF-8.

## 23.4 `std.process`

`Command` recebe programa e argumentos como vetor. Não concatena uma linha de shell. Isso reduz injeção e problemas de escape.

## 23.5 `std.time`

- `Instant`: timeout e duração, pois não volta no tempo.
- `SystemTime`: data civil e auditoria.
- `Duration`: intervalo com unidade.

## 23.6 `std.log`

Níveis: `trace`, `debug`, `info`, `warn` e `error`. Registros podem ser exibidos ou persistidos em formato estruturado. Dados secretos são removidos.

## 24. Memória e coleções

### 24.1 Alocadores

| Módulo                        | Estado | Função                        | Exemplo de uso      |
|-------------------------------|--------|-------------------------------|---------------------|
| <code>memory allocator</code> | E2     | contrato alloc/resize/free    | vector              |
| <code>memory arena</code>     | E2     | liberar muitos valores juntos | parser de manifesto |
| <code>memory mapping</code>   | E2     | mapear arquivo em memória     | índice grande       |
| <code>memory pool</code>      | E3     | reutilizar objetos fixos      | tarefas repetidas   |

Coleções que crescem recebem um alocador:

EXEMPLO: C LARP

```
get vector::Vector as items = vector::create(allocator);
defer vector::destroy(&items);
```

### 24.2 Catálogo de coleções

| Coleção                             | Estado | Operações             | Quando usar                     |
|-------------------------------------|--------|-----------------------|---------------------------------|
| <code>Span&lt;T&gt;</code>          | E2     | get, slice, length    | observar memória sem possuir    |
| <code>FixedVector&lt;T,N&gt;</code> | E2     | push, pop, get        | limite conhecido sem heap       |
| <code>Vector&lt;T&gt;</code>        | E2     | reserve, push, remove | sequência dinâmica              |
| <code>Stack&lt;T&gt;</code>         | E2     | push, pop, peek       | parsing e percurso              |
| <code>Queue&lt;T&gt;</code>         | E2     | enqueue, dequeue      | ordem de chegada                |
| <code>RingBuffer&lt;T&gt;</code>    | E2     | push, pop, capacity   | logs de tamanho fixo            |
| <code>BitSet</code>                 | E2     | set, clear, test      | conjuntos compactos             |
| <code>IntrusiveList&lt;T&gt;</code> | E3     | insert, remove        | evitar nó alocado separadamente |
| <code>HashMap&lt;K,V&gt;</code>     | E3     | insert, get, remove   | busca rápida sem ordem          |
| <code>BTreeMap&lt;K,V&gt;</code>    | E3     | insert, range, remove | dados ordenados                 |
| <code>Set&lt;T&gt;</code>           | E3     | add, contains, remove | valores únicos                  |
| <code>PriorityQueue&lt;T&gt;</code> | E3     | push, pop_min         | prioridade                      |
| <code>ObjectPool&lt;T&gt;</code>    | E3     | acquire, release      | objetos com limite              |

Cada coleção documenta complexidade, alocação, invalidação de ponteiros, falha ao atingir capacidade e thread safety.

### 24.3 Escolha prática

- `Array`: tamanho fixo faz parte do tipo.
- `Fatia` ou `Span`: função apenas observa dados.
- `FixedVector`: limite conhecido.
- `Vector`: tamanho cresce.
- `HashMap`: consulta por chave.
- `BTreeMap`: ordem e intervalos.
- `RingBuffer`: memória fixa e fluxo contínuo.

## 25. Dados, sincronização e Linux

### 25.1 Dados

| Módulo                      | Estado | Serve para                        |
|-----------------------------|--------|-----------------------------------|
| <code>data.bytes</code>     | E2     | ler campos de buffers com limites |
| <code>data.endian</code>    | E2     | little-endian e big-endian        |
| <code>data.utf8</code>      | E2     | validar e percorrer UTF-8         |
| <code>data.checksum</code>  | E3     | detectar corrupção acidental      |
| <code>data.serialize</code> | E3     | codificação limitada e versionada |

Checksum não substitui assinatura criptográfica.

### 25.2 Sincronização

| Módulo                   | Estado | Serve para                           |
|--------------------------|--------|--------------------------------------|
| <code>sync.atomic</code> | E3     | valores atômicos e ordens de memória |
| <code>sync.mutex</code>  | E3     | exclusão mútua com bloqueio          |
| <code>sync.rwlock</code> | E3     | vários leitores ou um escritor       |
| <code>sync.once</code>   | E3     | inicialização única                  |

A primeira ferramenta Bilarp deve ser simples e de processo único. Concorrência só entra quando houver necessidade medida.

### 25.3 Linux específico

| Módulo                       | Estado     | Quando usar                            |
|------------------------------|------------|----------------------------------------|
| <code>linux.syscall</code>   | E2 interno | construir wrappers da <code>std</code> |
| <code>linux.mmap</code>      | E2         | mapear arquivos e memória              |
| <code>linux.epoll</code>     | E3         | muitos descritores simultâneos         |
| <code>linux.signal</code>    | E3         | encerramento e sinais                  |
| <code>linux.mount</code>     | E3         | consultar ou montar com autorização    |
| <code>linux.namespace</code> | E3         | isolamento de processos                |
| <code>linux.netlink</code>   | E3         | estado do sistema e dispositivos       |
| <code>linux.xattr</code>     | E3         | atributos estendidos                   |

Aplicações comuns usam `std`. `linux.syscall` e wrappers com ponteiros formam fronteiras `unsafe`.

### 25.4 Hardware

Programas comuns executam em modo usuário. `unsafe` não concede privilégios. Dispositivos são acessados por `/dev`, `ioctl`, `sysfs`, `netlink`, bibliotecas e serviços autorizados.

Não haverá `hardware.*` na biblioteca padrão inicial. Recursos de baixo nível ficam em `experimental.linux.*` até possuírem caso de uso e testes.

## 26. SDK do Bilarp

| Módulo                           | Tipos                                                                           | Funcionalidades                   | Por que existe                   |
|----------------------------------|---------------------------------------------------------------------------------|-----------------------------------|----------------------------------|
| <code>bilarp.manifest</code>     | <code>Manifest</code> ,<br><code>Artifact</code> ,<br><code>Dependency</code>   | parse e validação                 | descrever pacotes com limites    |
| <code>bilarp.package</code>      | <code>PackageId</code> ,<br><code>Version</code> ,<br><code>PackageState</code> | consulta e comparação             | vocabulário comum                |
| <code>bilarp.repository</code>   | <code>Repository</code> ,<br><code>Channel</code> , <code>Index</code>          | refresh e busca                   | separar origem da aplicação      |
| <code>bilarp.plan</code>         | <code>UpdatePlan</code> ,<br><code>Change</code> , <code>Risk</code>            | resolver e explicar               | nenhuma mudança sem plano        |
| <code>bilarp.transaction</code>  | <code>Transaction</code> ,<br><code>Step</code> , <code>Journal</code>          | aplicar e verificar               | operação auditável               |
| <code>bilarp.snapshot</code>     | <code>Snapshot</code> ,<br><code>SnapshotBackend</code>                         | criar e restaurar                 | rollback quando real             |
| <code>bilarp.file_index</code>   | <code>FileOwner</code> ,<br><code>FileRecord</code>                             | localizar e verificar             | controle de arquivos             |
| <code>bilarp.policy</code>       | <code>Policy</code> , <code>Hold</code> ,<br><code>Rule</code>                  | avaliar e explicar                | decisão do usuário               |
| <code>bilarp.audit</code>        | <code>AuditEvent</code> ,<br><code>AuditStore</code>                            | registrar e consultar             | histórico confiável              |
| <code>bilarp.signature</code>    | <code>Signature</code> , <code>KeyId</code>                                     | verificar por biblioteca auditada | integridade e origem             |
| <code>bilarp.base_adapter</code> | <code>BaseAdapter</code> ,<br><code>BasePlan</code>                             | traduzir para a base              | reutilizar ecossistema existente |

### 26.1 Manifesto

Declara formato, pacote, versão, arquitetura, artefatos, hashes, dependências, conflitos, origem, canal, assinatura e limites. O parser nunca executa comandos.

### 26.2 Plano

Um plano lista instalação, atualização, downgrade, remoção e bloqueio. Depois de aprovado, torna-se imutável. Mudança no estado do sistema obriga nova validação.

### 26.3 Transação

EXEMPLO: TEXTO

```

created -> prepared -> applying -> verifying -> committed
                |               |
                v               v
                failed ----> reverting -> reverted

```

O journal é gravado antes de etapas irreversíveis. Rollback universal não é prometido: scripts, firmware ou serviços podem ter efeitos externos.

### 26.4 Índice de arquivos

Registra caminho, pacote responsável, hash esperado, metadados e última verificação. A primeira implementação utiliza armazenamento testado, como SQLite por FFI.



## 26.5 Política

Regras podem fixar versão, impedir downgrade, exigir aprovação, escolher canal, preservar arquivo local e exigir snapshot. Toda decisão informa qual regra foi aplicada.

# PARTE V - Especificação técnica

Uma nova etapa da apostila: avance no seu ritmo e consulte o glossário sempre que necessário.

## 27. Arquivos, módulos e gramática léxica

### 27.1 Estrutura de arquivo

Um arquivo possui, nesta ordem:

- 1 um namespace;
- 2 zero ou mais `use`;
- 3 declarações de tipos, constantes, globais e funções.

Vários arquivos podem compartilhar namespace. O manifesto define a raiz de fontes.

### 27.2 Importação

EXEMPLO: C LARP

```
use<std.io> as io;  
use<collections.vector> as vector;
```

`use<std.io>;` cria o apelido final `io`. Não existe importação curinga na v0.3. Colisões são erros.

### 27.3 Identificadores

- fonte UTF-8;
- identificadores ASCII no MVP;
- letras, números e `_`;
- primeiro caractere não numérico;
- distinção entre maiúscula e minúscula;

- chaves obrigatórias;
- ponto e vírgula em instruções simples.

## 27.4 Literais

EXEMPLO: C LARP

```
42
42u64
0xFFu32
0b1010_0101u8
0o755u32
3.14f64
true
false
'A'
b'A'
"texto UTF-8"
c"texto C terminado em zero"
null
```

Inteiro sem contexto usa `i32`; ponto flutuante usa `f64`. Valores maiores exigem tipo ou sufixo. `_` pode separar dígitos.

## 27.5 Palavras reservadas

EXEMPLO: TEXTO

```
as asm bi0 break case cast const continue default defer do else enum
fallthrough false finish fn for get global if loop namespace never null
public return struct switch true type union unsafe use volatile while
```

`return` permanece reservada; a sintaxe oficial é `finish`.

# 28. Semântica numérica e avaliação

## 28.1 Conversões

Não existe promoção numérica geral. Operandos precisam possuir o mesmo tipo, exceto literal contextual que caiba no destino. Conversão de sinal ou largura usa `cast`.

## 28.2 Ordem de avaliação

Expressões e argumentos são avaliados da esquerda para a direita. `&&` e `||` usam curto-circuito. Atribuição é instrução e não retorna valor.

## 28.3 Precedência

| Nível | Operadores                                                                                                                                | Associação |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------|------------|
| 1     | <code>()</code> , <code>[]</code> , <code>.</code> , <code>::</code>                                                                      | esquerda   |
| 2     | <code>!</code> , <code>~</code> , unário <code>-</code> , unário <code>+</code> , <code>*</code> , <code>&amp;</code> , <code>cast</code> | direita    |
| 3     | <code>*</code> , <code>/</code> , <code>%</code>                                                                                          | esquerda   |
| 4     | <code>+</code> , <code>-</code>                                                                                                           | esquerda   |
| 5     | <code>&lt;&lt;</code> , <code>&gt;&gt;</code>                                                                                             | esquerda   |
| 6     | <code>&lt;</code> , <code>&lt;=</code> , <code>&gt;</code> , <code>&gt;=</code>                                                           | esquerda   |
| 7     | <code>==</code> , <code>!=</code>                                                                                                         | esquerda   |
| 8     | <code>&amp;</code> binário                                                                                                                | esquerda   |
| 9     | <code>^</code>                                                                                                                            | esquerda   |
| 10    | <code> </code> binário                                                                                                                    | esquerda   |
| 11    | <code>&amp;&amp;</code>                                                                                                                   | esquerda   |
| 12    | <code>  </code>                                                                                                                           | esquerda   |

## 28.4 Falhas verificadas

- divisão por zero chama `panic`;
- deslocamento inválido chama `panic`;
- overflow comum chama `panic`;
- índice inválido chama `panic`;
- modo release preserva o mesmo contrato.

# 29. Memória, ponteiros e `unsafe`

## 29.1 Armazenamento

| Categoria | Início                       | Fim                            |
|-----------|------------------------------|--------------------------------|
| local     | entrada no bloco             | saída do bloco                 |
| global    | carregamento do processo     | término do processo            |
| heap      | alocação explícita           | liberação explícita            |
| mapping   | <code>mmap</code> ou wrapper | <code>munmap</code> ou wrapper |

Não existe garbage collector.

## 29.2 Ponteiros

EXEMPLO: C LARP

```
*u8
*const u8
*volatile u32
```

Ponteiros não carregam tamanho, validade ou lifetime. O otimizador assume que ponteiros brutos podem criar alias. `restrict` não entra no MVP.

## 29.3 Operações `unsafe`

Exigem `unsafe`:

- desreferenciar ponteiro bruto;
- fazer aritmética de ponteiro;
- converter ponteiro e inteiro;
- ler campo inativo de `union`;
- chamar função externa não encapsulada;
- executar syscall bruta com ponteiros;
- ignorar verificação de limites;
- usar Assembly.

EXEMPLO: C LARP

```
unsafe {  
    get *u8 as pointer = cast<*u8>(address);  
    get u8 as value = *pointer;  
}
```

## 29.4 Limite da garantia

O código fora de `unsafe` é chamado “verificado”, não “memory safe”. Sem ownership e lifetimes, continuam possíveis use-after-free, double free, fatias pendentes e corridas de dados.

## 29.5 `volatile` e atômicos

`volatile` representa acesso observável, útil em interfaces específicas. Não substitui sincronização. Atômicos possuem ordens `relaxed`, `acquire`, `release`, `acq_rel` e `seq_cst` em E3.

# 30. Erros e limpeza técnica

## 30.1 Resultados concretos

Antes de genéricos:

EXEMPLO: C LARP

```
struct OpenResult {  
    get bool as ok;  
    get File as file;  
    get i32 as code;  
}
```

Depois:

EXEMPLO: C LARP

```
result<File, FsError>  
optional<PackageId>
```

## 30.2 Contrato de `panic`

- nunca retorna;
- não desenrola a pilha;
- não executa `defer` durante a falha;

- tenta emitir mensagem mínima;
- termina o processo por abort;
- mantém semântica em todas as otimizações.

### 30.3 `defer`

Executa em saídas normais causadas por `finish`, `break` e `continue` que atravessem o bloco. A ordem é inversa à declaração.

## 31. FFI, ABI e atributos

### 31.1 Alvo inicial

- `x86_64-unknown-linux-gnu`;
- System V AMD64;
- arquivos ELF;
- ABI C para fronteiras estáveis.

### 31.2 Importar C

EXEMPLO: C LARP

```
@extern("C", "puts")
unsafe fn i32 c_puts(get *const u8 as text);
```

### 31.3 Exportar para C

EXEMPLO: C LARP

```
@export("C", "bilarp_add")
public fn i32 add(get i32 as a, get i32 as b) {
    finish to a + b;
}
```

### 31.4 Atributos

| Atributo                | Estado | Uso                      |
|-------------------------|--------|--------------------------|
| <code>@extern</code>    | E2     | importar símbolo externo |
| <code>@export</code>    | E2     | exportar símbolo estável |
| <code>@repr("C")</code> | E2     | layout compatível com C  |
| <code>@align(N)</code>  | E2     | exigir alinhamento       |
| <code>@section</code>   | E3     | escolher seção ELF       |
| <code>@used</code>      | E3     | preservar símbolo        |
| <code>@inline</code>    | E3     | solicitar inline         |
| <code>@no_inline</code> | E3     | impedir inline           |
| <code>@test</code>      | E2     | marcar teste             |
| <code>@when</code>      | E2     | compilação condicional   |

`@entry`, `@interrupt`, `@naked` e layout `packed` saem da especificação ativa porque não criaremos kernel e ainda não há caso hospedado comprovado.

## 31.5 Compilação condicional

EXEMPLO: C LARP

```
@when(target.arch == "x86_64")
fn bool has_fast_path() {
    finish to true;
}
```

Propriedades: `target.arch`, `target.os`, `target.abi`, `target.endian` e `target.pointer_width`. Não existe préprocessador textual `#define`.

## 31.6 Assembly

Assembly embutido é E4. Entradas, saídas, registradores destruídos e efeitos de memória devem ser declarados. Em Linux, instruções privilegiadas continuam proibidas ao processo comum.

# PARTE VI - Construindo o compilador e o produto

Uma nova etapa da apostila: avance no seu ritmo e consulte o glossário sempre que necessário.

## 32. Arquitetura do compilador

### 32.1 Bootstrap em Rust

O primeiro compilador será escrito em Rust. Isso facilita AST, diagnósticos e testes e reduz erros de memória no próprio compilador. Executáveis C Larp não dependerão do runtime Rust.

Autocontenção vem depois. Não é possível escrever o primeiro compilador inteiramente em uma linguagem que ainda não pode ser compilada.

### 32.2 Pipeline

EXEMPLO: TEXTO



```

arquivo .clarp
|
lexer: caracteres -> tokens
|
parser: tokens -> AST
|
resolução: nomes e módulos
|
semântica: tipos, retorno e unsafe
|
IR tipada da C Larp
|
LLVM IR
|
LLVM: otimização e x86-64
|
objeto ELF
|
LLD: executável Linux

```

## 32.3 O que cada parte faz

| Parte        | Explicação simples         | Responsabilidade técnica         |
|--------------|----------------------------|----------------------------------|
| lexer        | separa palavras e símbolos | tokens com spans                 |
| parser       | reconhece a estrutura      | AST e recuperação de erro        |
| resolver     | encontra cada nome         | escopo, módulo e visibilidade    |
| type checker | confere tipos              | conversões e retornos            |
| verifier     | aplica regras extras       | inicialização, limites e unsafe  |
| IR           | simplifica o programa      | contrato independente da sintaxe |
| LLVM backend | produz código nativo       | ABI, registradores e objeto      |
| driver       | coordena ferramentas       | CLI, cache e linker              |

## 32.4 Por que LLVM

LLVM já resolve alocação de registradores, seleção de instruções, relocação, informação de depuração e otimização. Um backend próprio completo atrasaria o uso real da linguagem por anos.

O projeto continua controlando sintaxe, tipos, verificações, IR, biblioteca, diagnósticos e FFI. Um emissor NASM pequeno poderá existir como ferramenta didática, mas não será o backend principal.

## 32.5 Diagnóstico profissional

EXEMPLO: TEXTO

```

erro[E021]: ponteiro bruto usado fora de unsafe
--> src/main.clarp:8:13
|
8 |     value = *pointer;
|           ^^^^^^^ a validade não pode ser provada
|
= ajuda: use um wrapper ou `unsafe { ... }`

```

O compilador deve continuar após erros recuperáveis para mostrar mais de um problema, sem produzir uma cascata inútil.

## 33. Ferramenta de terminal

EXEMPLO: TEXTO

```
clarp check arquivo.clarp
clarp build arquivo.clarp -o programa
clarp run arquivo.clarp
clarp test
clarp fmt arquivo.clarp
clarp emit ast arquivo.clarp
clarp emit typed-ir arquivo.clarp
clarp emit llvm-ir arquivo.clarp
clarp emit asm arquivo.clarp
clarp emit obj arquivo.clarp
clarp targets
clarp version
```

### 33.1 Manifesto

EXEMPLO: TOML

```
[package]
name = "bilarp_inspect"
version = "0.1.0"
source = "src"

[build]
target = "x86_64-unknown-linux-gnu"
entry = "bilarp_inspect.main"
panic = "abort"

[dependencies]
std = "bundled"
```

O MVP aceita biblioteca empacotada e dependências locais. Registro remoto fica para depois.

### 33.2 Perfis

| Perfil  | Finalidade                                        |
|---------|---------------------------------------------------|
| debug   | símbolos e diagnóstico completo                   |
| release | otimização com mesma semântica                    |
| test    | descoberta de <code>@test</code> e instrumentação |

## 34. Plano de implementação

### 34.1 Marco 0 - corpus oficial

- 20 programas válidos;
- 20 inválidos com erro esperado;
- precedência aprovada;
- conversões aprovadas;
- ABI dos primitivos;
- estrutura do repositório e CI.

### 34.2 Marco 1 - lexer

Tokens, literais, comentários, linhas e colunas. Fuzzing não pode derrubar o lexer.

### 34.3 Marco 2 - parser

AST de funções, declarações, blocos e expressões. Recuperação de erro.

### 34.4 Marco 3 - semântica

Escopos, tipos, retorno, inicialização e `unsafe`.

### 34.5 Marco 4 - executável

Typed IR, LLVM IR, objeto e LLD. Aceitação:

EXEMPLO: C LARP

```
namespace acceptance.exit_42;

bi0 main() {
    finish to 42;
}
```

O executável precisa terminar com código 42.

### 34.6 Marco 5 - FFI e saída mínima

Chamar C, ser chamado por C e escrever uma linha.

### 34.7 Marco 6 - programas úteis

Arrays, fatias, structs, arquivos e erros. Prova: `hexdump`, leitor ELF ou contador de linhas.

### 34.8 Marco 7 - memória, genéricos e manifesto

Arena, vector, result e parser limitado de manifesto.

### 34.9 Marco 8 - Bilarp somente leitura

Consultar pacote, versão e arquivos pela distribuição-base sem modificar nada.

### 34.10 Marco 9 - planejamento

Políticas, dry-run e plano imutável explicável.

## 34.11 Marco 10 - transação

Journal, snapshot quando suportado, aplicação em máquina virtual, verificação e falhas injetadas.

## 34.12 Marco 11 - autocontenção

Reescrever partes do compilador em C Larp e executar bootstrap reproduzível.

## 34.13 Escala de tempo

Lexer e parser podem ser prototipados em meses. Um compilador para ferramentas reais costuma exigir muitos meses ou anos. Biblioteca madura, Bilarp transacional e autocontenção são metas de vários anos para uma equipe pequena.

# 35. Como evitar um toy model

---

Um compilador pequeno é real quando conclui o fluxo inteiro e possui evidências. Ele ainda pode ser experimental, mas não é fictício.

## 35.1 Provas necessárias

- 1 Produzir executável ELF válido.
- 2 Obedecer à ABI System V.
- 3 Chamar C e ser chamado por C.
- 4 Compilar vários arquivos.
- 5 Possuir testes positivos e negativos.
- 6 Nunca travar por código inválido conhecido.
- 7 Executar fuzzing de lexer e parser.
- 8 Reproduzir builds.
- 9 Compilar ferramentas úteis.
- 10 Documentar limitações.

## 35.2 Programas de prova

- calculadora de checksum;
- hexdump simplificado;
- leitor de cabeçalho ELF;
- contador de linhas;
- alocador arena;
- parser de manifesto;
- inventário Bilarp somente leitura.

## 35.3 Testes do compilador

| Tipo      | Exemplo                              |
|-----------|--------------------------------------|
| lexer     | string sem fechamento                |
| parser    | chave ausente                        |
| semântico | retorno com tipo errado              |
| negativo  | ponteiro fora de <code>unsafe</code> |
| ABI       | função C com inteiros e struct       |
| execução  | saída e código de retorno            |
| regressão | caso mínimo de cada bug              |

## 35.4 Apresentação honesta

Descrição correta durante E0 e E1:

“C Larp é uma linguagem experimental para programas de sistemas no Linux x86-64. O compilador está em desenvolvimento e não deve ser usado em produção.”

Evitar afirmações como:

- “totalmente segura”;
- “substitui Assembly”;
- “controle total de hardware” em modo usuário;
- “pronta para criar sistemas operacionais”;
- “mais rápida que C” sem benchmark;
- “distribuição pronta” sem imagem e testes.

## 36. Gramática orientativa

Esta EBNF descreve o núcleo estrutural. Expressões seguem a tabela de precedência.

EXEMPLO: EBNF

```

program      = namespace_decl, { use_decl }, { declaration } ;

namespace_decl = "namespace", qualified_name, ";" ;
use_decl      = "use", "<", qualified_name, ">",
[ "as", identifier ], ";" ;
qualified_name = identifier, { ".", identifier } ;

declaration  = { attribute },
( const_decl
| global_decl
| type_alias
| struct_decl
| enum_decl
| union_decl
| function_decl
| main_decl ) ;

attribute    = "@", identifier,
[ "(", [ argument_list ], ")" ] ;

main_decl    = "bin0", "main", "(", ")", block ;

function_decl = [ "public" ], [ "unsafe" ],
"fn", type, identifier,
"(", [ parameter_list ], ")",
( block | ";" ) ;

parameter_list = parameter, { ",", parameter } ;
parameter      = "get", type, "as", identifier ;

const_decl    = "const", type, identifier,
"=", expression, ";" ;
global_decl   = "global", variable_decl ;
variable_decl = "get", type, "as", identifier,
[ "=", expression ], ";" ;

type_alias    = "type", identifier, "=", type, ";" ;
type          = qualified_name
| "*", [ "const" ], [ "volatile" ], type
| "[", [ integer_literal ], "]", type ;

block         = "{", { statement }, "}" ;
statement     = variable_decl
| assignment_statement
| expression, ";"
| block
| if_statement
| while_statement
| do_while_statement
| for_statement
| loop_statement
| switch_statement
| unsafe_statement
| finish_statement
| defer_statement
| "break", ";"
| "continue", ";"
| "fallthrough", ";" ;

finish_statement = "finish", [ "to", expression ], ";" ;
unsafe_statement = "unsafe", block ;
defer_statement  = "defer", expression, ";" ;

```

`bin0` e `bout1` não aparecem porque pertencem à biblioteca de I/O.

## 37. Exercícios

---

### 37.1 Fundamentos

- 1 Escreva um programa que termine com código 7.
- 2 Declare duas variáveis `i32` e devolva a soma.
- 3 Crie uma constante para o número máximo de tentativas.
- 4 Explique a diferença entre `char`, `u8` e `str`.
- 5 Corrija uma condição que usa `=` em vez de `==`.

### 37.2 Decisões e laços

- 1 Classifique um valor como negativo, zero ou positivo.
- 2 Conte de 0 a 9 com `for`.
- 3 Percorra um array e conte elementos maiores que 10.
- 4 Use `continue` para ignorar valores zero.
- 5 Use `break` ao encontrar o primeiro valor negativo.

### 37.3 Funções e estruturas

- 1 Crie `fn i32 min(a, b)`.
- 2 Modele `Package` com nome, versão e estado.
- 3 Crie enum para níveis de log.
- 4 Escreva função que recebe fatia e devolve o maior valor.
- 5 Explique por que a função precisa tratar fatia vazia.

### 37.4 Design

- 1 Escreva uma história de usuário para inspeção de arquivo.
- 2 Separe CLI, serviço e adaptador.
- 3 Liste erros esperados.
- 4 Defina três invariantes.
- 5 Crie um plano de testes com caso de falha.

### 37.5 Engenharia

- 1 Explique por que `unsafe` não concede privilégio Linux.
- 2 Compare `Vector` e `FixedVector`.
- 3 Explique por que LLVM será usado.
- 4 Liste os critérios do executável que retorna 42.
- 5 Desenhe o pipeline do compilador com suas próprias palavras.

## 38. Referência rápida

### 38.1 Declarações

EXEMPLO: C LARP

```
get i32 as value = 0;
const i32 LIMIT = 10;
global get u64 as counter = 0;
```

### 38.2 Funções

EXEMPLO: C LARP

```
fn i32 add(get i32 as a, get i32 as b) {
    finish to a + b;
}

fn void show() {
    finish;
}
```

### 38.3 Controle

EXEMPLO: C LARP

```
if (condition) { }
else { }

while (condition) { }
do { } while (condition);
for (get usize as i = 0; i < n; i += 1) { }
loop { }
```

### 38.4 Tipos

EXEMPLO: TEXTO

```
i8 i16 i32 i64 i128
u8 u16 u32 u64 u128
isize usize byte
f32 f64 bool char str void never
*T *const T *volatile T
[N]T []T []const T
```

### 38.5 Organização

EXEMPLO: C LARP

```
namespace project.module;
use<std.io> as io;
public fn i32 api();
```

### 38.6 Segurança delimitada

EXEMPLO: C LARP

```
unsafe {
    value = *pointer;
}
```



## 39. Glossário

| Termo             | Explicação                                                   |
|-------------------|--------------------------------------------------------------|
| ABI               | regras binárias para chamadas, pilha, registradores e layout |
| alocador          | componente que concede e libera memória dinâmica             |
| API               | interface pública oferecida por um módulo                    |
| AST               | árvore que representa a estrutura do código                  |
| backend           | parte do compilador que gera código nativo                   |
| bootstrap         | construir o primeiro compilador com outra linguagem          |
| byte              | unidade de 8 bits                                            |
| código verificado | código submetido às verificações disponíveis                 |
| compilador        | programa que transforma fonte em executável                  |
| dependência       | módulo ou pacote usado por outro                             |
| distribuição      | kernel Linux mais programas, pacotes e políticas             |
| ELF               | formato de executável e objeto do Linux                      |
| FFI               | chamada entre linguagens diferentes                          |
| frontend          | lexer, parser e análise semântica                            |
| função            | bloco reutilizável com parâmetros e retorno                  |
| invariante        | condição que sempre precisa permanecer verdadeira            |
| IR                | representação intermediária do compilador                    |
| lifetime          | período em que armazenamento permanece válido                |
| linker            | ferramenta que combina objetos e símbolos                    |
| LLD               | linker do ecossistema LLVM                                   |
| LLVM              | infraestrutura usada para gerar código nativo                |
| manifesto         | documento estruturado que descreve pacote e artefato         |
| módulo            | conjunto organizado de tipos e funções                       |
| MVP               | menor versão que comprova o fluxo completo                   |
| namespace         | endereço lógico que organiza nomes                           |
| ownership         | modelo que define responsabilidade por recurso               |
| ponteiro          | valor que contém endereço de memória                         |
| runtime           | suporte executado junto do programa                          |
| snapshot          | estado recuperável de volume ou sistema de arquivos          |
| syscall           | solicitação de serviço ao kernel Linux                       |
| transação         | sequência planejada, registrada e confirmada                 |
| <b>unsafe</b>     | região com invariantes assumidas pelo programador            |

## 40. Checklist do projeto

---

### Antes do lexer

- ☐ Aprovar sintaxe e precedência.
- ☐ Aprovar tipos e conversões.
- ☐ Aprovar overflow, limites e panic.
- ☐ Definir ABI dos tipos.
- ☐ Criar corpus válido e inválido.
- ☐ Fixar versões de Rust, LLVM e LLD.
- ☐ Criar CI e harness de testes.

### Antes da biblioteca

- ☐ Executável ELF real.
- ☐ FFI em duas direções.
- ☐ Diagnósticos estáveis.
- ☐ Alocação e erro documentados.
- ☐ Testes de regressão.

### Antes do Bilarp modificar o sistema

- ☐ Escolher distribuição-base.
- ☐ Implementar ferramenta somente leitura.
- ☐ Definir manifesto e ameaça.
- ☐ Integrar assinatura auditada.
- ☐ Implementar dry-run e plano.
- ☐ Testar em máquina virtual.
- ☐ Implementar journal.
- ☐ Verificar suporte real a snapshot.
- ☐ Injetar falhas e testar recuperação.

# 41. Conclusão

---

A C Larp começa com códigos simples: valores, decisões, laços e funções. Depois cresce para arquivos, FFI, memória explícita e ferramentas de sistema. A mesma linguagem poderá escrever componentes do Bilarp quando o compilador e a biblioteca atingirem maturidade suficiente.

O Bilarp será uma distribuição baseada no Linux, não um kernel novo. Sua contribuição será dar ao usuário um plano compreensível, políticas claras, histórico e controle transacional sobre atualizações e arquivos, aproveitando o ecossistema da distribuição-base.

A credibilidade virá de resultados verificáveis: um executável real, uma ABI obedecida, testes, programas úteis e documentação honesta. O próximo artefato é o corpus de conformidade; o próximo código é o lexer; a primeira entrega pública será pequena e verdadeira.